

```
[1]: a=[1,2,3,4,5]
      print(a)
```

[1, 2, 3, 4, 5]

```
[2]: import numpy as np
      arr=np.array([1,2,3,4,5])
      print(arr)
```

[1 2 3 4 5]

```
[4]: a=[1,2,3]
      b=[4,5,6]
      res=[]
      for i in range(len(a)):
          res.append(a[i]+b[i])
      print(res)
```

[5, 7, 9]

```
[6]: import numpy as np
      a=np.array([1,2,3])
      b=np.array([4,5,6])
      print(a+b)
      print(a-b)
      print(a*b)
      print(a**b)
```

[5 7 9]
[-3 -3 -3]
[4 10 18]
[1 32 729]

```
[31]: import numpy as np
```

```
[31]: import numpy as np
arr=np.array([1,2,3,4,5])
ar=np.array([[1,2,3],[4,5,6]])
a=np.array([[[1,2],[3,4],[5,6]]])
print(arr)
print(ar)
print(a)
print(arr.shape)
print(ar.shape)
print(a.shape)
print(arr.ndim)
print(ar.ndim)
print(a.ndim)
print(arr.size)
print(ar.size)
print(a.size)
print(a.dtype)
s=np.array([[
    [[1,2],[3,4]],
    [[5,6],[7,8]],
    [[8,9],[2,9]]]])
print(s.shape)
```

```
[1 2 3 4 5]
[[1 2 3]
 [4 5 6]]
[[[1 2]
  [3 4]
  [5 6]]]
(5,)
(2, 3)
(1, 3, 2)
1
2
3
```

```
(2, 3)
(1, 3, 2)
1
2
3
5
6
6
int32
(1, 3, 2, 2)
```

Array creation in numpy

```
[24]: import numpy as np
a=np.array([1,2,3,4,5])
b=np.zeros(5)
print(a)
print(b)
c=np.zeros((2,2))
print(c)
d=np.ones(3)
print(d)
e=np.ones((2,3))
print(e)
```

```
[1 2 3 4 5]
[0. 0. 0. 0. 0.]
[[0. 0.]
 [0. 0.]]
[1. 1. 1.]
[[1. 1. 1.]
 [1. 1. 1.]]
```

```
[37]: import numpy as np
a=np.array([1,2,3,4,5])
b=np.zeros(5)
```

```
[37]: import numpy as np
a=np.array([1,2,3,4,5])
b=np.zeros(5)
print(a)
print(b)
print(np.arange(1,10,2))
print(np.linspace(0,20,5))
print(np.linspace(0,20,3))
print(np.random.rand(5))
print(np.random.randint(1,100,7))
print(np.random.randn(5))
```

```
[1 2 3 4 5]
[0. 0. 0. 0. 0.]
[1 3 5 7 9]
[ 0.  5. 10. 15. 20.]
[ 0. 10. 20.]
[0.96561778 0.87803834 0.68898292 0.84395932 0.60683836]
[78 18 52 35 38 20 28]
[ 0.86910963 -2.06475724  0.94023053 -0.30109393  0.32428226]
```

numpy indexing and slicing

```
[58]: import numpy as np
a=np.array([10,20,30,43,50,60,67])
#1d array indexing
print(a[2])
b=np.array([[1,2,3],
            [4,5,6],
            [7,8,9]])
#2d array indexing
print(b[1][1])
#1d slicing
```

```
[58]: import numpy as np
a=np.array([10,20,30,43,50,60,67])
#1d array indexing
print(a[2])
b=np.array([[1,2,3],
            [4,5,6],
            [7,8,9]])
#2d array indexing
print(b[1][1])
#1d slicing
print(a[2:7])
print(a[:4])
print(a[::2])
print(a[::1])
#2d slicing
print(b[1])
print(b[:,1]) # it print 1 st column
print(a[::-2])
print(b[:, -1])
print(a[a>50])
print(a[a<50])
print(a[[0,4,3,6]]) # fancyindexing selecting mulitple specific values
print(a[a%2==0])
a[4]=1000
print(a)
```

```
30
5
[30 43 50 60 67]
[10 20 30 43]
[10 30 50 67]
[10 20 30 43 50 60 67]
[4 5 6]
[2 5 8]
[67 50 30 10]
[2 6 0]
```

```
30
5
[30 43 50 60 67]
[10 20 30 43]
[10 30 50 67]
[10 20 30 43 50 60 67]
[4 5 6]
[2 5 8]
[67 50 30 10]
[3 6 9]
[60 67]
[10 20 30 43]
[10 50 43 67]
[10 20 30 50 60]
[ 10 20 30 43 1000 60 67]
```

mathematical operation in numpy

```
[61]: import numpy as np
a=np.array([1,2,3])
print(a+10) # scalar broadcasting
b=np.array([[1,2,3],
            [4,5,6]])
print(b+10) # 2d broadcasting
print(b+a) # row broadcasting dimension should be equal
# always no of columns must be same
```

```
[11 12 13]
[[11 12 13]
 [14 15 16]]
[[2 4 6]
 [5 7 9]]
```

matrix operation

```
[66]: import numpy as np
a=np.array([[1,2],
            [3,4]])
b=np.array([[4,3],
            [2,1]])
print(a*b)
print(np.dot(a,b))
print(a@b)
print(np.matmul(a,b))
print(a.T) # row become column and column become row
```

```
[[4 6]
 [6 4]]
[[ 8  5]
 [20 13]]
[[ 8  5]
 [20 13]]
[[ 8  5]
 [20 13]]
[[1 3]
 [2 4]]
```

```
[70]: # universal mathematical function
import numpy as np
a=np.array([4,9,16])
print(np.sqrt(a))
b=np.array([-5,-6,-2,-8])
print(np.abs(b))
print(np.log(a))
print(np.exp(a))
```

```
[2. 3. 4.]
```

```
[2. 3. 4.]
[5 6 2 8]
[1.38629436 2.19722458 2.77258872]
[5.45981500e+01 8.10308393e+03 8.88611052e+06]
```

```
[73]: # round function
arr=np.array([1.234,5.674])
print(np.round(arr,2)) # after decimal only 2 values
print(np.floor(arr)) # lower value
print(np.ceil(arr)) # higher value
```

```
[1.23 5.67]
[1. 5.]
[2. 6.]
```

```
[95]: #statistical function
import numpy as np
a=np.array([10,2,3,40,50])
print(np.mean(a))
print(np.median(a)) # after sorting only the middle value can be taken
print(np.std(a))
print(np.max(a))
print(np.min(a))
print(np.percentile(a,25))
print(np.percentile(a,50))
print(np.percentile(a,75))
print(np.percentile(a,100))
print(np.var(a))
print(np.argmin(a)) # minimum index value
print(np.argmax(a)) # maximum index value
b=np.array([1,2,3,4,5])
c=np.array([2,4,6,8,10])
print(np.corrcoef(b,c)) # return positive or neg relationships
d=np.array([1,2,3,4,5])
e=np.array([10,8,6,4,2])
print(np.corrcoef(d,e))
```

```
print(np.argmax(a)) # maximum index value
b=np.array([1,2,3,4,5])
c=np.array([2,4,6,8,10])
print(np.corrcoef(b,c)) # return positive or neg relationships
d=np.array([1,2,3,4,5])
e=np.array([10,8,6,4,2])
print(np.corrcoef(d,e))
#aggregate function

d=np.array([[1,3,4],
            [5,6,8]])
print(np.sum(d))
print(np.sum(d,axis=0))
print(np.sum(d,axis=1))
```

```
21.0
10.0
20.039960079800558
50
2
3.0
10.0
40.0
50.0
401.6
1
4
[[1. 1.]
 [1. 1.]]
[[ 1. -1.]
 [-1.  1.]]
27
[ 6  9 12]
[ 8 19]
```

problems

```
[107]: import numpy as np
scores=np.array([[60,70,80],
                 [50,90,95],
                 [70,80,40]])
print(np.mean(scores))
print(np.max(scores,axis=1)) # row
normalize=(scores-np.min(scores))/(np.max(scores)-np.min(scores))
print(normalize)
print(np.mean(scores,axis=0)) #column
```

```
70.55555555555556
[80 95 80]
[[0.36363636 0.54545455 0.72727273]
 [0.18181818 0.90909091 1.         ]
 [0.54545455 0.72727273 0.         ]]
[60.         80.         71.66666667]
```

```
[111]: import numpy as np
arr=np.array([10,20,30,40,50])
s=arr[arr>25]
print(s)
total=sum(s)
print(total)
```

```
[30 40 50]
120
```

```
[ ]: import numpy as np
arr=np.array([1,
              [2],
              [3]])
```